

Senior architecture review

# From LangChain to multi-agent systems

How our agent stack actually fits together — from a single chained call, to a stateful graph, to two independent agents that hand work to each other. Grounded in the notebook and pipelines we've already run, not slideware.

9 sections

Live code, not pseudocode

Internal models: gpt-oss-120b-ITG · qwen · mistral

01 · LangChain

02 · LangGraph

03 · CrewAI

04 · Comparison

05 · First notebook

06 · Multi-model agents

07 · Fintech case study

08 · Multi-agent deep dive

09 · Decision guide

## 01 LangChain — linear composition

A library for wiring one LLM call into the next, in a fixed order

LangChain gives you the building blocks for composing calls to a language model into a pipeline: format a prompt, call the model, parse

what comes back, optionally feed that into the next step. Prompt templates, output parsers, a light "chain" abstraction — all designed for workflows that run straight through, start to finish, in one fixed order.

The defining trait isn't the library, it's the shape of the workflow: once step 2 has run, the pipeline never goes back to step 1. Every input takes the identical path.

#### ANALOGY — A SINGLE-LINE FACTORY

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#123B2B'}}
graph LR
  A["🔧 Chassis"] --> B["⚙️ Engine"] --> C["🎨 Paint"] --> D["✅ QA"]
```

Once a chassis leaves station 1, it never returns to station 1. Every car takes the same four stations, in the same order, every time. That fixed, one-directional sequence is exactly what a LangChain chain is.

#### Reach for LangChain when

The workflow is genuinely straight-line: prompt → parse → next prompt → done. Nothing needs to be revisited, and the next step never depends on a runtime decision about what just happened.

---

## 02 LangGraph — stateful, cyclical orchestration

Same building blocks, plus a graph that can branch and loop

LangGraph keeps the same primitives — prompts, models, parsers — but replaces the fixed sequence with an explicit graph: named **nodes**, a

shared **state** object every node reads and writes, and **edges** that can be fixed or conditional. The structural difference that matters: a graph can cycle. A node's output can route back to an earlier node. A chain cannot do that — there's no "earlier" to return to.

Same factory, two upgrades. After the chassis station, we now branch: an EV order routes to a battery station, a petrol order routes to an engine station. And QA can now fail a car and send it back to paint for rework, instead of shipping a defect.

#### ANALOGY — THE SAME FACTORY, WITH A BRANCH AND A REWORK LOOP

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#123B2B'}}
graph TD
  A["🔧 Chassis"] --> B{"EV or petrol?"}
  B -->|EV| C["🔋 Battery station"]
  B -->|Petrol| D["⚙️ Engine station"]
  C --> E["🎨 Paint"]
  D --> E
  E --> F{"QA pass?"}
  F -->|"No - rework"| E
  F -->|Yes| G["🚗 Ship car"]
```

The branch at "EV or petrol?" is a **conditional edge**. The loop from "QA pass?" back to Paint is a **cycle** — a plain chain cannot express either of these; LangGraph's StateGraph can.

#### Reach for LangGraph when

The workflow needs to loop back, retry, branch on a runtime decision, or pause for a human to approve a step. Everything from Section 5 onward in this reference is built on LangGraph, not LangChain, for exactly this reason.

---

## 03 CrewAI — role-based agent crews

A different starting point: describe roles, not a graph

CrewAI starts from a different question. Instead of drawing the graph yourself, you describe **agents** by role — a goal, a backstory, a set of tools — and hand them a task. The framework manages collaboration and hand-offs between agents largely implicitly, optimized for standing up a "team" of agents with far less orchestration code than a hand-wired graph.

**When we'd reach for it:** fast prototyping of a role-based crew — a Researcher, a Writer, an Editor — where the exact execution path doesn't need to be pinned down in code, and trading some control for speed of setup is an acceptable trade.

**When we would not:** anywhere in a regulated workflow — like the loan underwriting or support escalation case studies ahead — where we need to know, deterministically and provably, which path an application took and why. CrewAI's implicit hand-offs make that harder to guarantee than LangGraph's explicit conditional edges and typed state.

### Not used in our examples

Every case study in this reference is built on LangGraph. CrewAI is included here for completeness and so the comparison in Section 04 is an informed choice, not a gap.

---

## 04 Framework comparison

The same decision, laid out side by side

|                        | LangChain                         | LangGraph                                               | CrewAI                                  |
|------------------------|-----------------------------------|---------------------------------------------------------|-----------------------------------------|
| Structure              | Linear chain                      | Graph — nodes + edges                                   | Role-based crew                         |
| Cycles / loops         | No                                | Yes                                                     | Limited, implicit                       |
| Conditional branching  | No                                | Yes, explicit<br>( <code>add_conditional_edges</code> ) | Implicit, via agent reasoning           |
| State                  | Passed step to step               | Shared, typed, inspectable                              | Managed internally by the framework     |
| Auditability           | High, but only for straight lines | High — every hop is a named node                        | Lower — hand-offs aren't pinned in code |
| Best for               | Simple prompt pipelines           | Regulated, branching, multi-step workflows              | Fast multi-agent prototypes             |
| Used in this reference | Conceptual foundation             | Sections 05 – 08                                        | Not used                                |

---

## 05 Hands-on — our first LangGraph notebook

The exact notebook we ran together: `Agentic_Demo1_Internal_LLM.ipynb`

Seven short steps, no third-party LLM SDK — just `requests` against our internal gateway. This is the smallest possible LangGraph program: one state schema, one node, one edge in, one edge out. Every case study later in this reference is this same shape, just with more nodes and more models.

### 1 • Setup

Load config from `.env`. No API SDK — just two environment variables.

```
cell [1] 0.3s

from dotenv import load_dotenv
import os

load_dotenv()

if os.environ.get("INTERNAL_LLM_BASE_URL") is None:
    raise ValueError("INTERNAL_LLM_BASE_URL is not set in the environm
if os.environ.get("INTERNAL_LLM_API_KEY") is None:
    raise ValueError("INTERNAL_LLM_API_KEY is not set in the environm
```

### 2 • The internal model client

`ChatInternalLLM` — a minimal client with `retry/backoff`, and `AIMessage`, a stand-in for LangChain's own message type since `.content` is all this notebook needs.

```
cell [2] 2.7s

class AIMessage:
    """Minimal stand-in for LangChain's AIMessage — just enough for `
    which is all the rest of this notebook actually uses."""
    def __init__(self, content: str):
```

```

        self.content = content

    def __repr__(self):
        return f"AIMessage(content={self.content!r})"

class ChatInternalLLM:
    def __init__(
        self,
        model: str = "gpt-oss-120b",
        temperature: float = 0.7,
        max_tokens: int = 512,
        base_url: str | None = None,
        api_key: str | None = None,
        timeout: float = 60.0,
        max_retries: int = 3,
    ):
        self.model = model
        self.temperature = temperature
        self.max_tokens = max_tokens
        self.base_url = (base_url or os.environ["INTERNAL_LLM_BASE_URI"])
        self.api_key = api_key or os.environ["INTERNAL_LLM_API_KEY"]
        self.timeout = timeout
        self.max_retries = max_retries
        self._session = requests.Session()

    def invoke(self, prompt: str) → AIMessage:
        url = f"{self.base_url}/chat/completions"
        headers = {
            "Authorization": f"Bearer {self.api_key}",
            "Content-Type": "application/json",
        }
        payload = {
            "model": self.model,
            "messages": [{"role": "user", "content": prompt}],
            "temperature": self.temperature,
            "max_tokens": self.max_tokens,
        }

        last_error = None
        for attempt in range(1, self.max_retries + 1):
            try:
                response = self._session.post(url, headers=headers, json=payload)

```

```

        response.raise_for_status()
        data = response.json()
        content = data["choices"][0]["message"]["content"]
        return AIMessage(content)
    except (requests.RequestException, KeyError, IndexError) as exc:
        last_error = exc
        if attempt < self.max_retries:
            time.sleep(min(2 ** attempt, 8)) # 2s, 4s, 8s backoff
            continue

    raise RuntimeError(
        f"Failed to reach internal LLM gateway at {url} (model={self.model!r}, "
        f"after {self.max_retries} attempts: {last_error})"
    ) from last_error

def __repr__(self):
    return f"ChatInternalLLM(model={self.model!r}, base_url={self.base_url!r})"

```

### 3 • Instantiate the model and try it once

```

cell [3] - [4] 3.4s

llm = ChatInternalLLM(model="gpt-oss-120b", temperature=0.7, max_tokens=1000)
llm
>>> ChatInternalLLM(model='gpt-oss-120b', base_url='https://api.llm.com/v1')

llm.invoke("what is water")

```

### 4 • The graph — state, node, and the welcome function

```

cell [5] 0.0s

from typing import TypedDict

class graph_schema(TypedDict):
    name: str
    message: str

def welcome(state: graph_schema) → graph_schema:

```

```

current_name = state["name"]
current_message = state["message"]
response = llm.invoke(f"My name is {current_name}. {current_message}")
print(f"your message is: {current_message}")
print(f"llm response is: {response}")
state['message'] = response
return state

```

## 5 • Wire up the graph

cell [6] 21.9s

```

from langgraph.graph import StateGraph, START, END

graph = StateGraph(graph_schema)
graph.add_node("welcome", welcome)
graph.add_edge(START, "welcome")
graph.add_edge("welcome", END)

```

## 6 • Visualize it

`draw_mermaid()` prints the same diagram language every diagram in this document uses — the graphs in Sections 07 and 08 aren't a different tool, they're this exact method, drawn by hand for readability.

cell [7] 0.9s

```

from IPython.display import display, Markdown

first_graph = graph.compile()
mermaid_diagram = first_graph.get_graph().draw_mermaid()
print(mermaid_diagram)

```

OUTPUT — THE GRAPH `DRAW_MERMAID()` ACTUALLY PRODUCED

```

%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#123B2B'
graph TD

```

```
s((START)) → welcome["welcome  
(gpt-oss-120b)"]  
welcome → e((END))
```

## 7 • Run it

```
cell [8] 1.0s  
  
first_graph.invoke({"name": "Meenakshi Sundaram", "message": "Hello, I  
  
your message is: Hello, how are you?  
llm response is: Hello Meenakshi! I'm doing great, thanks for asking.  
  
{'name': 'Meenakshi Sundaram',  
  'message': "Hello Meenakshi! I'm doing great, thanks for asking. How
```

---

## 06 Multi-model agents — the core idea

One extra rule on top of everything in Section 02

A multi-model agent is nothing more than the LangGraph from Section 02, with one additional rule: **the model is a per-node choice, not a global setting**. Each node picks the internal model best suited to its job — small and fast for mechanical work, largest and most capable reserved for the step where being wrong is expensive — and every node reads and writes the same shared state.

THREE NODES, THREE DELIBERATE MODEL CHOICES

```
%{init: {'theme': 'base', 'themeVariables': {'primaryColor': '#123B2B'}}
```

```
graph LR
  A["Extract  
small, fast model"] → B["Reason  
mid-size model"] → C["Judge  
largest model"] → D["shared state"]
```

#### Rule of thumb

Send extraction and classification to the cheapest model that stays accurate. Reserve the largest model for the step with the highest cost of being wrong.

## 07 Case study 1 — fintech loan underwriting

Single agent, four nodes, three internal models, one conditional branch

A loan pre-screening pipeline that never sends applicant data outside the network: every model call goes to an internally-hosted deployment. Intake and risk scoring always run; compliance and the final decision only run on the happy path — a high-risk application is routed to manual review instead.

#### PIPELINE ARCHITECTURE

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#123B2B'}}
graph TD
  Intake["Application Intake  
Mistral"] → Risk["Risk Analysis  
Qwen"]
  Risk → |risk_score ≤ 70| Compliance["Compliance Check  
GPT-0SS"]
  Risk → |"risk_score > 70"| Manual["Manual Review"]
```

```

(workshop: + Fraud Agent)"]
  Compliance → Decision["Decision Node
rule-based"]
  Manual → Decision
  Decision → Out(["Approve / Reject / Manual Review"])

```

| Node          | Model                | Why this model                                        |
|---------------|----------------------|-------------------------------------------------------|
| Intake        | <code>mistral</code> | Fast, cheap — mechanical field extraction             |
| Risk analysis | <code>qwen</code>    | Step-by-step reasoning for a scored, explained output |
| Compliance    | <code>gpt-oss</code> | Highest cost of being wrong — our largest model       |
| Decision      | — (rule-based)       | Deterministic, auditable aggregation, no model call   |

#### Workshop extension — add a fraud detection agent

The team's workshop task takes this exact pipeline and inserts a 5th node between Risk and Manual Review: a fraud-check that runs only when `risk_score > 70`, and can override the final decision to `HOLD_FOR_INVESTIGATION`. Six numbered TODOs in `workshop_starter.py` guide the change — the reference implementation in `multi_model_fintech_agent.py` is the comparison baseline.

---

## 08 Case study 2 — multi-agent support escalation

Two independent LangGraph agents. No orchestrator — Agent A calls Agent B directly.

This is the step beyond Section 07: instead of one agent with one graph, there are **two independently-built agents**, each with its own state schema and its own multi-node graph. Agent A owns the decision to escalate *and* the hand-off itself — there's no third orchestrator file wiring them together. Agent A's `escalate_to_specialist_node` simply imports Agent B's already-compiled graph and calls it, exactly like calling any other Python function.

### FULL ARCHITECTURE — BOTH AGENTS, SIX NODES, THE DIRECT CALL

```
%%{init: {'theme':'base', 'themeVariables': {'primaryColor':'#123B2B'}
graph TD
    subgraph AgentA["AGENT A — Tier-1 Support Triage"]
        A1["classify_intent
mistral-small-2506-ITG"] --> A2["retrieve_kb
bge-large-en-v1.5-ITG"] --> A3["draft_reply
qwen3.6-27b"]
        A3 --> A4{"confidence < 50 or
sentiment negative?"}
        A4 -->|No| A5["resolve
rule-based"]
    end
    subgraph AgentB["AGENT B — Specialist Escalation"]
        B1["tag_severity
mistral-small-4-ITG"] --> B2["root_cause
gpt-oss-120b-ITG"] --> B3["draft_resolution
qwen-latest"]
    end
    A4 -->|"Yes — direct call
agent_b_app.invoke()"| B1
    B3 -->|"returns EscalationState"| A6["final_response
back in Agent A"]
```

```
A5 → Out(["response to customer"])
A6 → Out
```

Agent B has no knowledge Agent A exists — it takes an `EscalationState` in and returns one. That's what keeps the two agents independently buildable and independently testable (both are verified standalone below).

| Agent                     | Node             | Model                             |
|---------------------------|------------------|-----------------------------------|
| A — Tier-1 Support Triage | classify_intent  | mistral-small-2506-ITG            |
|                           | retrieve_kb      | bge-large-en-v1.5-ITG (embedding) |
|                           | draft_reply      | qwen3.6-27b                       |
| B — Specialist Escalation | tag_severity     | mistral-small-4-ITG               |
|                           | root_cause       | gpt-oss-120b-ITG                  |
|                           | draft_resolution | qwen-latest                       |

## The hand-off, in code

From `agents/agent_a_support_triage/nodes.py` — the entire "multi-agent" mechanism is these two lines.

```
agents/agent_a_support_triage/nodes.py
```

```
from agents.agent_b_specialist_escalation.graph import app as agent_b
from agents.agent_b_specialist_escalation.state import EscalationState
```

```
def escalate_to_specialist_node(state: dict) → dict:
    agent_b_input: EscalationState = {
        "customer_message": state["customer_message"],
        "tier1_intent": state["intent"],
        "tier1_draft": state["draft_reply"],
```

```

        "tier1_confidence": state["confidence"],
        "kb_matches": state["kb_matches"],
        "trace": [],
    }
    result = agent_b_app.invoke(agent_b_input)    # ← plain function

    return {
        "escalated": True,
        "final_response": result["resolution"],
        "resolved_by": "specialist (Agent B)",
        "trace": [f"[Agent A / escalate] confidence below threshold -
    }

```

## Sample run — both paths, one invocation of Agent A

Actual output from `python main.py`. Case 2's trace shows all seven hops — four in Agent A, three in Agent B — as a single combined audit log.

```
$ python main.py - CASE 2
```

```

[Agent A / classify_intent / mistral-small-2506-ITG] intent=billing_d:
[Agent A / retrieve_kb / bge-large-en-v1.5-ITG] top match=KB-104: Dupl
[Agent A / draft_reply / qwen3.6-27b] confidence=38
[Agent A / escalate] confidence 38 < 50 (or negative sentiment) -- ha
[Agent B / tag_severity / mistral-small-4-ITG] severity=high -- Repeat
[Agent B / root_cause / gpt-oss-120b-ITG] Duplicate charge caused by a
[Agent B / draft_resolution / qwen-latest] Refund the duplicate charge

resolved_by: "specialist (Agent B)"

```

### Why direct call, not a third orchestrator

Agent A already owns the decision of *whether* to escalate — a conditional edge on its own state. Adding a third graph just to route "if escalate, call Agent B" would duplicate that decision. The direct call keeps ownership of the hand-off with the agent that decided to make it, and keeps Agent B a plain, independently-testable graph with zero awareness of who calls it.

---

## 09 Decision guide

One table, for the next design review

| Your workflow looks like...                                                                          | Reach for                                                 |
|------------------------------------------------------------------------------------------------------|-----------------------------------------------------------|
| One prompt in, one parsed answer out, always the same order                                          | LangChain                                                 |
| Multiple steps, some need to branch or retry, all owned by one team                                  | LangGraph — single agent                                  |
| Two or more workflows that are independently owned and only sometimes need to hand off to each other | LangGraph — multi-agent, direct call (Section 08 pattern) |
| Many hand-offs, orchestration logic itself needs central visibility/control                          | LangGraph — multi-agent, parent orchestrator graph        |
| Fast prototype of a role-based team, control and audit trail not yet a requirement                   | CrewAI                                                    |